



Testing basics. Devops testing tools

What Is Testing in DevOps?

DevOps is a progressive method of development and deployment that simplifies and streamlines processes among different team silos, enabling developers, operation specialists, QA engineers and analysts to cooperate better and deal with tight deadlines.

Another crucial feature of modern quality assurance is automation. Today, it is typical to build software in short sprint cycles, and it would be impossible to run a large number of the required control cases manually. In DevOps QA testing, automation is a must for a successful performance, as it reduces the number of repetitive tasks and shortens the time needed to obtain quality checking results. Automation is the biggest quality maintainer and speed promoter.

What Is Testing in DevOps?

The role and skills of testers have also been reshaped. Today, QA testers transform into automation engineers who possess more technical skills, understand both application functionality and its technical environment, know the basics of coding and are able to analyze it. They know how to utilize automation tools and create scripts in parallel with development.

Unlike before, a tester is not an isolated member, but rather an essential part of the team who is involved in the process from the very beginning. Automation engineers ensure quality during the whole process and validate the right functioning of the developed application.

Continuous Testing Benefits

There are four basic ongoing processes in the DevOps culture:

- Continuous integration
- Continuous delivery
- Continuous testing
- Continuous monitoring

All of them are heavily intertwined. Continuous delivery and integration are impossible without controlling and monitoring since they need quick assessments of changes for frequent high-quality releases.

Continuous Testing Benefits

Software testing for DevOps is an integral part that has some clear benefits, as it:

- Provides fast and continuous feedback
- Excludes errors at early stages
- Enhances the checking process efficiency
- Accelerates the rate of change
- Allows flawless continuity throughout SDLC (system development life cycle)
- Allows developers to bring new features to market faster
- Ensures the safety of the system
- Allows managers to make better trade-off decisions

As DevOps breaks the barriers between Dev and Ops and encourages better coordination among various silos, it ensures that everyone takes responsibility for the quality. Therefore, checking is embedded along

Developing a DevOps Testing Strategy

The adoption of DevOps in your company will definitely have a powerful impact on control strategies. As it's impossible to automate everything, you need an automation strategy to get maximum ROI from time and money spent. A well-planned strategy can increase the speed of development and free up teams to concentrate on more essential tasks. Here is a so-called DevOps testing tutorial that will help you develop the right automation strategy.

Developing a DevOps Testing Strategy

Choose the Right Cases to Automate

What cases do you choose for automation: repetitive tests, high-risk cases, large data sets or checks for different browsers and environments? It depends on the service you are developing and on your team's capabilities Your task is to automate the cases that will bring the most benefits for the development process and the whole organization.

Implement Automation Throughout a Sprint

Speedy releases can be achieved only if the development and examination are finished simultaneously at the end of the sprint. That's why quality assurance should begin as early as possible. Keyword-based tests can be written before the development of new functionalities, as they correlate with the task but don't go into detail. During the process, an automated unit testing takes action, writes scripts as soon as the code is available and runs continuously.

Developing a DevOps Testing Strategy

Continue to Apply Automated Cases

Created cases can become complicated and irrelevant after a while, which is why you should consider building flexible tests. The best solution is to write small cases without overloading them with dozens of steps beginning from the top. Instead, break apps into smaller steps and check them separately.

The second way to create automated tests that will last long is to make them independent on the UI. Don't include the names of the interface elements in the script, as they may change while the development moves on. Instead, use the backend functions to keep test cases relevant over time and protect them from inevitable changes.

Strategic Test Planning: Phases and Methodologies

- **Requirements analysis:** Understand the project scope, objectives, and user requirements to establish testing goals and priorities.
- **Test strategy development:** Define the overall testing approach, including types of testing, resources, timelines, and risk management strategies.
- **Test case design:** Develop comprehensive test cases based on requirements, user stories, and test scenarios to ensure adequate coverage.
- **Test environment setup:** Prepare the necessary infrastructure, tools, and resources for executing tests in various environments.
- **Test execution:** Execute test cases systematically, record results, and monitor for defects, ensuring thorough coverage of functional and non-functional aspects.
- **Defect management:** Track and prioritize defects, collaborate with developers for resolution, and verify fixes through retesting.
- **Test closure:** Evaluate testing outcomes, document lessons learned, and prepare test reports for stakeholders.

Crafting Effective Test Cases

- **Clear objectives:** Define clear objectives for each test case, including the specific functionality or scenario being tested.
- **Test case structure:** Structure test cases logically, with a clear description of inputs, expected outputs, and preconditions.
- **Boundary conditions:** Include test cases that explore boundary conditions, edge cases, and invalid inputs to uncover potential issues.
- **Positive and negative testing:** Design test cases to validate both expected (positive) and unexpected (negative) behavior of the system.
- **Data variation:** Test with a variety of data inputs to ensure robustness and reliability across different scenarios.
- **Reusability:** Create reusable test cases to maximize efficiency and maintainability of the testing process.
- **Traceability:** Ensure each test case is traceable to specific requirements or user stories to demonstrate coverage.
- **Prioritization:** Prioritize test cases based on risk, criticality, and business impact to focus testing efforts effectively.

Bug Lifecycle: Reporting and Tracking Defects

- **Bug Identification:** Identify and document defects encountered during testing or user feedback.
- **Bug Reporting:** Report defects using a standardized format, including detailed descriptions, steps to reproduce, and associated screenshots or logs.
- **Bug Prioritization:** Prioritize defects based on severity, impact on functionality, and business priorities.
- **Bug Assignment:** Assign defects to appropriate team members, such as developers or QA engineers, for resolution.
- **Bug Resolution:** Developers analyze, reproduce, and resolve defects, updating the bug status accordingly.
- **Bug Verification:** QA engineers verify bug fixes to ensure they effectively address the reported issue.
- **Bug Closure:** Once verified, defects are closed, and relevant documentation is updated.
- **Bug Analysis:** Periodically analyze defect data to identify trends, root causes, and areas for improvement in the development process.

Test Automation: Advantages and Challenges

- Test automation streamlines testing processes, offering benefits such as increased efficiency, improved accuracy, and enhanced test coverage. Automated tests detect defects early, leading to cost savings over time. However, implementing automation requires initial investment and ongoing maintenance. Selecting suitable tests for automation, choosing appropriate tools, and managing false positives/negatives pose challenges. Despite these challenges, effective test automation can significantly improve software quality and development efficiency.

Introduction to Test Automation Tools

- Test automation tools facilitate the creation, execution, and management of automated tests. These tools come with various features and capabilities to support different testing needs. Common features include test script creation, test case management, reporting, and integration with development and CI/CD pipelines. Test automation tools range from open-source frameworks like Selenium and Appium to commercial solutions such as HP UFT and Tricentis Tosca. Each tool has its strengths and suitability for specific testing scenarios, platforms, and technologies. By leveraging test automation tools, organizations can streamline testing processes, improve efficiency, and deliver high-quality software products reliably





Types of testing

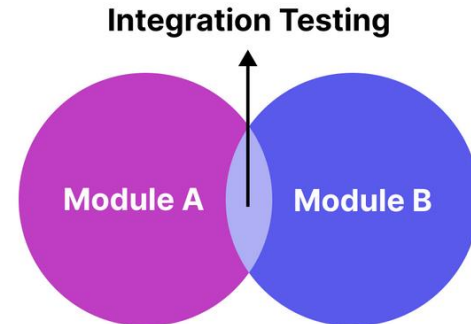
Unit Testing Essentials



- Unit testing verifies the functionality of individual units or components of software in isolation. It ensures that each unit behaves as expected according to its design. By writing and executing test cases for these units independently, developers can catch bugs early and streamline debugging and maintenance processes. Unit testing is integral to DevOps practices, enhancing software quality and enabling seamless integration and delivery pipelines.

Exploring Integration Testing

- Integration testing evaluates the interaction between different units or components of software. It ensures that these units integrate smoothly and function correctly together as a whole system. By testing the interfaces and interactions between units, integration testing identifies any issues arising from their integration. This type of testing is crucial for validating the behavior of the system as a whole and ensuring that all components work together seamlessly. Integration testing is an essential part of the software development lifecycle and plays a vital role in achieving robust and reliable software products.



System Testing Overview



- System testing evaluates the complete and integrated software system to ensure that it meets specified requirements and functions correctly in its intended environment. Unlike unit and integration testing, which focus on individual components or their interactions, system testing assesses the entire system's behavior. It validates various aspects such as functionality, performance, security, and reliability. System testing is crucial for uncovering defects that may arise from the interactions between different components or subsystems. By conducting comprehensive system tests, organizations can verify that their software meets user expectations and delivers the intended value.

Understanding Acceptance Testing



- Acceptance testing, also known as user acceptance testing (UAT), assesses whether a software system meets the business requirements and is acceptable for delivery to end-users. It involves testing the system's functionality, usability, and overall performance against predefined criteria or user stories. Acceptance testing typically occurs towards the end of the software development lifecycle and is conducted by stakeholders or end-users themselves. The primary goal of acceptance testing is to ensure that the software fulfills its intended purpose and meets the needs of its users effectively. Successful acceptance testing indicates readiness for deployment and user adoption, validating the software's alignment with business objectives.



Testing Levels

Delving into Component Testing

- Component testing, also known as module testing or unit testing, focuses on verifying the individual units or components of a software system. It involves testing each unit in isolation to ensure that it performs as intended according to its design specifications. Component testing is typically performed by developers during the early stages of the development process. By thoroughly testing each component, developers can detect and rectify defects early, thus contributing to overall software quality and reducing the risk of issues during integration and system testing phases. Component testing is an essential practice for ensuring the reliability and robustness of software components before they are integrated into larger systems.

Exploring Interface Testing

- Interface testing focuses on verifying the interactions and communication between different software components or systems. It ensures that interfaces, such as APIs, web services, and user interfaces, function correctly and exchange data as expected. Interface testing verifies the compatibility, reliability, and integrity of interfaces to ensure seamless integration between various software modules or external systems. This type of testing helps identify any inconsistencies or errors in data exchange, ensuring that the software components can communicate effectively with each other. Interface testing is crucial for validating the interoperability of software systems and ensuring smooth operation in real-world environments.

Comprehensive End-to-End Testing

- End-to-end testing evaluates the entire software system's functionality and performance in a simulated real-world environment. It involves testing the system from start to finish, including all integrated components, subsystems, and external dependencies. End-to-end testing aims to validate the system's behavior and workflow, ensuring that it meets user requirements and business objectives. By simulating typical user scenarios, end-to-end testing helps identify any potential issues or bottlenecks that may arise during actual usage. This level of testing provides valuable insights into the system's reliability, scalability, and usability, ultimately ensuring a seamless user experience.



Testing Techniques

Black Box Testing Unveiled

- Black box testing, also known as behavioral testing or functional testing, assesses the software's functionality without considering its internal code structure. Testers examine the system's inputs and outputs, as well as its external behavior, to validate its compliance with specified requirements. This technique treats the software as a black box, focusing solely on its observable behavior and functionality from the user's perspective. Black box testing enables testers to identify defects, errors, and inconsistencies in the software's behavior without needing access to its internal implementation details. By simulating various user scenarios, black box testing helps ensure that the software meets user expectations and performs as intended in different usage contexts.

White Box Testing Unveiled

- White box testing, also known as structural testing or glass box testing, examines the internal structure and implementation of the software code. Testers analyze the software's internal logic, paths, and structure to design test cases that ensure comprehensive code coverage. Unlike black box testing, which focuses on external behavior, white box testing delves into the software's internal workings, including control flow, data flow, and conditional statements. This technique enables testers to uncover errors, flaws, and vulnerabilities hidden within the codebase. By assessing the code's integrity and correctness, white box testing helps improve code quality, identify potential security vulnerabilities, and enhance overall software reliability.

Grey Box Testing Unveiled

- Grey box testing combines elements of both black box and white box testing approaches, offering a balanced perspective on software testing. Testers have partial knowledge of the internal workings of the software while also evaluating its external behavior. This technique allows testers to design test cases based on a combination of user-driven scenarios and insights into the software's internal structure. Grey box testing enables testers to uncover defects and vulnerabilities that may arise from both external interactions and internal code logic. By leveraging this hybrid approach, grey box testing offers a comprehensive assessment of the software's functionality, performance, and security, contributing to enhanced software quality and reliability.



Devops testing tools

QuerySurge

QuerySurge is the smart data testing solution that is the first-of-its-kind full DevOps solution for continuous data testing.



Key Features

- Robust API with 60+ calls
- Seamlessly integrates into the DevOps pipeline for continuous testing
- Verifies large amounts of data quickly
- Validates difficult transformation rules between multiple source and target systems
- Detects requirements and code changes, updates tests accordingly and alerts team members of said changes
- Provides detailed data intelligence & data analytics

Jenkins

Jenkins is an open-source DevOps testing tool. It can be used to automate all types of tasks such as building, testing, and deploying software. This tool allows developers to quickly find and solve defects in their code base to automate testing of their builds.



Jenkins

Key Features

- Provides support to scale out to a large number of nodes and distribute the workload equally
- Compatible with all OS and versions of Linux, Mac OS or Windows
- It offers easy installation as Jenkins comes as a WAR file. All user needs to do is to drop WAR into your JEE container and your setup up ready to run.
- Jenkins can be easily set-up and configured with the help of its web interface
- It's can easily distribute work across several machines

Bamboo

Bamboo is a Continuous Integration tool which is also used for DevOps testing. It performs the automatic build, test, and releases in a single place. It supports many technologies and languages like Docker, Git, SVN, Mercurial, and Amazon S3 buckets.



Key Features

- Setting up Bamboo is pretty simple
- Run parallel batch tests
- Per-environment permissions feature allows developers and QA to deploy to their environments
- It can trigger builds based on changes detected in the repository, push notifications from Bitbucket

Jmeter

Apache JMeter is an open source load testing tool. It is designed to measure the performance of websites. This load testing tool can be used in DevOps methodology.



Key Features

- JMeter allows performing load and performance test for various server types.
- This load testing tool store its test plans in XML format which allows users to generate the test plan using a text editor.
- The tool can be used to perform automated and functional testing of the applications.

Selenium

Selenium is the most popular automated testing tool. It is specifically designed to support automation testing of a wide range of browsers.

Key Features

- Support for parallel test execution reduces the time taken in test execution
- Selenium needs very lesser resources when compared to other testing tools.
- Test cases prepared using this testing tool can be executed on any OS
- It supports popular programming languages like Java, Python, C#, Perl, PHP, and JavaScript.



Appium

Appium is an open source automation tool for mobile applications. It allows users to test all kind of native, mobile, web, and hybrid apps. It also supports the automated tests on emulators and simulators. It is actively used in DevOps process.



Key Features

- Easy setup process.
- It is a simple application which needs a very little memory of for test process.
- Testing Native apps do not need SDK. It offers standard automation APIs which can be used on for all types of platforms.

Soap UI

SoapUI is a cross-platform free open source API testing tool for SOAP and REST. It is widely used for Devops testing tool to perform functional and load testing on API.



Key Features

- The GUI of the software is easy to handle and use
- Vulnerability testing feature helps to secure website from hackers and viruses.
- It is possible to do the detailed analysis using its reporting feature.
- SQL Injection feature provides some standard SQL queries and methods to identify the weak areas of the application.

CruiseControl



CruiseControl is a Continuous Integration Tool. Its web interface provides details of the current and previous builds. It also provides support for Remote Management.

Key Features

- Integration with different Source Control systems like csv, svn, git, hg, perforce, clearcase, filesystem, etc.
- It allows building multiple projects on single server
- It allows seamless Integration with other external tools like NAnt, NDepend, NUnit, MBUnit and Visual Studio

Vagrant



Vagrant is a DevOps testing tool. It used for building and managing virtual machine environments in a single workflow. It offers easy-to-use workflow and focuses on automation. It also reduces development environment setup time and increases production parity.

Key Features

- Simple, easy to use, and powerful tool
- Vagrant integrates with existing configuration management tools like Chef, Puppet, Ansible, or Salt
- Vagrant works seamlessly on as OS Mac, Linux, and Windows
- No complicated setup process only download and install
- Create a single file for projects for describing the type of machine and software that users want to install

PagerDuty

PagerDuty is a DevOps tool that helps businesses to enhance their brand reputation. It is an incident management solution supporting CI strategy. This DevOps testing tool helps teams to deliver high-performing apps.

Key Features

- Provide Real-time alerts
- Reliable & Rich Alerting facility
- Event Grouping & Enrichment
- Gain visibility into critical systems and applications
- Easily detect and resolve incidents from development through production
- It offers Real-Time Collaboration System & User Reporting
- It supports Platform Extensibility
- It allows scheduling & Automated Escalations
- Full-stack visibility across development and production environments
- Event intelligence for actionable insights



Snort



Snort is powerful free, open-source tool that helps in the detection of intruders. It also highlights malicious attacks against the system. It allows real-time traffic analysis and packet logging.

Key Features

- Performs protocol analysis and content searching
- It allows signature-based detection of attacks by analysing packets
- It offers real-time traffic analysis, packet logging detects, buffer overflows, etc.

Docker



Docker is a DevOps technology suite. It allows DevOps teams to build, ship, and run distributed applications. This tool allows users to assemble apps from components and work collaboratively. This open source platform for managing containers of an app as a single group.

Key Features

- CaaS Ready platform running with built in orchestration
- Flexible image management with a private registry to store and manage images and configure image caches
- Isolates apps in containers to eliminate conflicts for enhancing security

Puppet Enterprise



Puppet enterprise tool eliminates manual work for software delivery process. It helps developer to deliver great software rapidly

Key Features

- Puppet enterprise tool eliminates manual work for software delivery process. It helps developer to deliver great software rapidly
- Model and manage your entire environment
- Intelligent orchestration and visual workflows
- Real-time context-aware reporting
- Define and continually enforce infrastructure
- It inspects and reports on packages running across infrastructure
- Desired state conflict detection and remediation



UpGuard analyzes data about the networks to the device in a single numerical value. It gives a quick sense of security risk.

Key Features

- UpGuard helps businesses around the world to gain visibility into their technology
- This DevOps tool allows increasing in speed of software delivery. It is accomplished through the automation of numbers of processes and technologies.
- It allows users to trust a third-party with sensitive data
- The procedures used to govern assets are as important as the configurations themselves



AppVerify is the continuous testing solution designed for business applications. AppVerify allows you to test the entire business flow, including all third-party apps, without adding plugins or writing a single line of code. It facilitates continuous testing for Agile and DevOps Teams.

Key Features

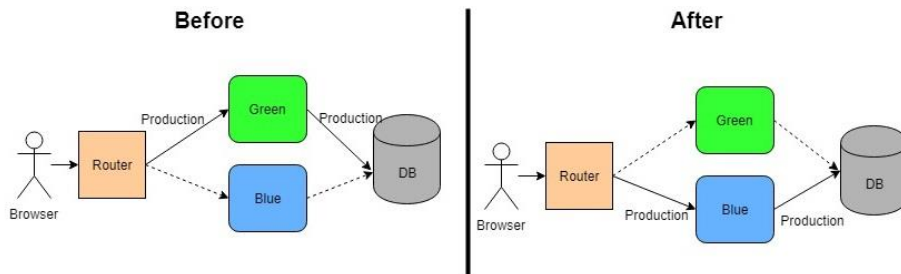
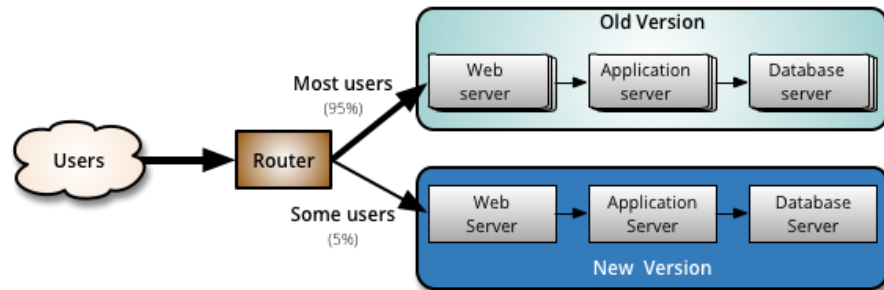
- Test any application by reproducing the same user experience from all your access points: Thin, fat clients, and web portals
- AppVerify replicates the users' interactions with your application and gives you valuable metrics about the end users' experience including screenshots of failures.
- Scripts are created automatically when you use your application and can be easily edited without coding.
- They can then be seamlessly applied to performance testing and application monitoring modules available.



Advanced

Testing in Production

- **Canary Releases:** In a canary release, a small subset of users or traffic is directed to the new version of the software while the majority continues to use the stable version. This allows teams to monitor the performance and gather feedback before gradually rolling out the update to all users.
- **Blue-Green Deployments:** Blue-green deployments involve maintaining two identical production environments: one active (blue) and one inactive (green). The new version of the software is deployed to the inactive environment and tested extensively. Once validated, traffic is switched from the old environment to the new one, minimizing downtime and risk.



AI and Machine Learning in Testing: Current Applications and Future Prospects



- AI and machine learning are increasingly being integrated into testing processes to enhance efficiency and effectiveness. Current applications include test case generation, predictive analytics for defect identification, automated test maintenance, and intelligent test optimization. These technologies enable faster test execution, improved accuracy in defect detection, and better utilization of testing resources. In the future, AI and machine learning are expected to further revolutionize testing by enabling autonomous testing, self-healing test suites, and advanced anomaly detection. By embracing AI and machine learning in testing, organizations can stay ahead of the curve, deliver higher quality software, and meet evolving customer demands effectively.



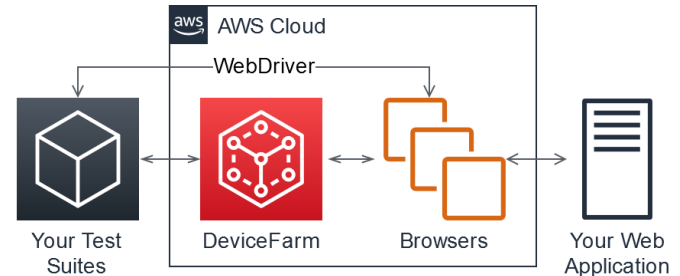
test.ai



functionize

Virtualization and Cloud Testing: Tools and Platforms

- Virtualization and cloud testing leverage cloud computing resources to perform testing activities efficiently and cost-effectively. Platforms like Amazon Web Services (AWS) and Microsoft Azure offer a range of services and tools tailored for testing purposes. With AWS, testers can leverage services such as Amazon EC2 for scalable virtualized testing environments, AWS Device Farm for mobile app testing, and AWS Lambda for serverless testing execution. Similarly, Azure provides offerings like Azure Virtual Machines for flexible testing environments, Azure DevTest Labs for managing test environments, and Azure App Service for web app testing. By utilizing these platforms, organizations can achieve greater flexibility, scalability, and cost savings in their testing efforts, ultimately leading to improved software quality and faster time-to-market.



DevSecOps: Integrating Security into DevOps

sonarqube

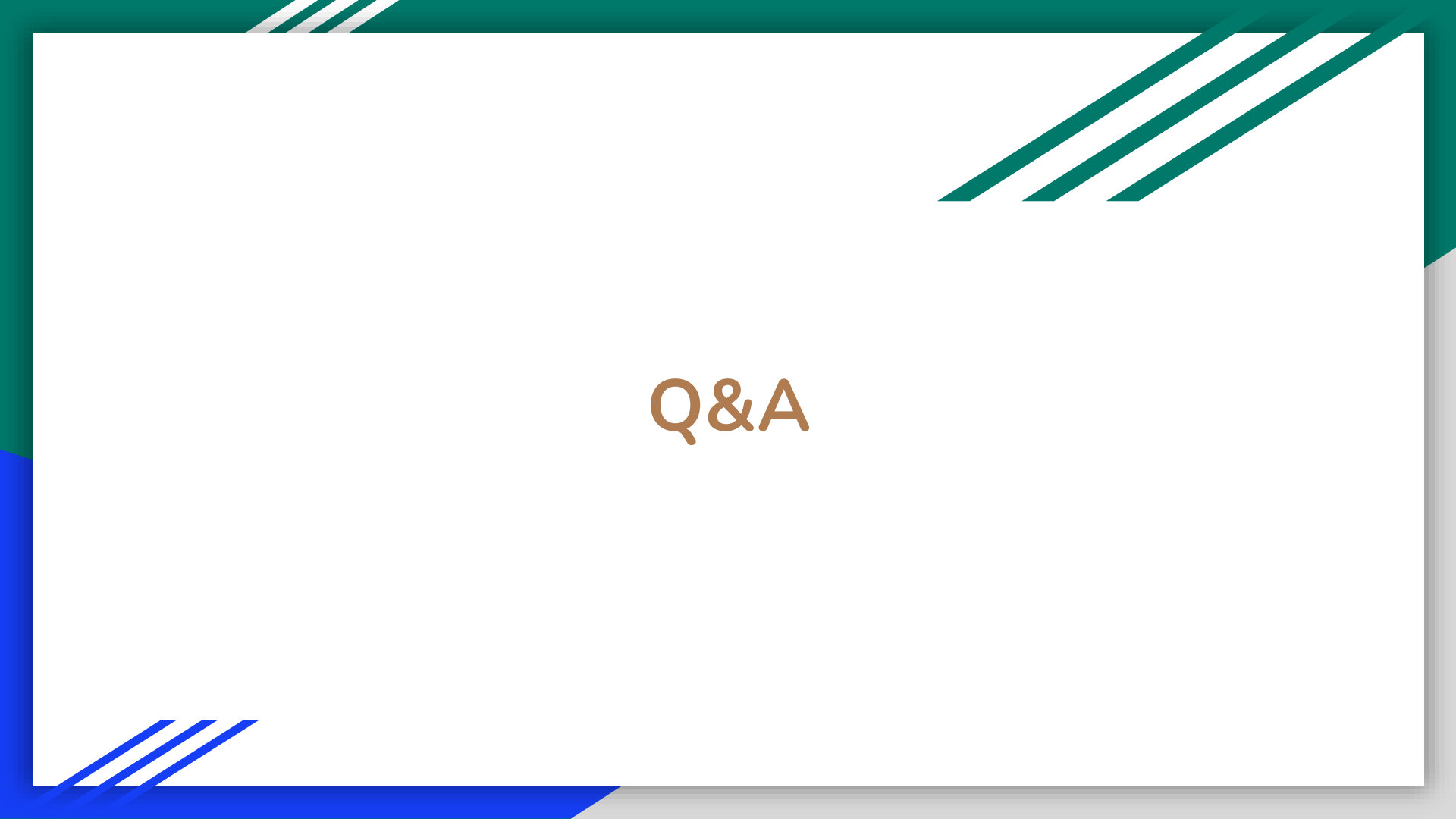


DevSecOps emphasizes integrating security practices into every stage of the DevOps pipeline, including testing. In this context, testing plays a crucial role in identifying and addressing security vulnerabilities early in the software development lifecycle. By incorporating security testing into automated CI/CD pipelines, DevSecOps ensures that security measures are applied consistently and efficiently throughout the development process. This approach includes:

- **Automated Security Testing:** Integrating automated security testing tools and techniques, such as static application security testing (SAST), dynamic application security testing (DAST), and interactive application security testing (IAST), into the CI/CD pipeline to identify security flaws in the codebase.
- **Security Code Reviews:** Conducting regular security-focused code reviews as part of the code review process to identify and remediate security vulnerabilities in the application code.
- **Vulnerability Scanning:** Performing regular vulnerability scans on dependencies, libraries, and third-party components used in the application to detect and remediate known security vulnerabilities.
- **Penetration Testing:** Conducting penetration tests to simulate real-world attacks and identify potential security weaknesses in the application infrastructure and configurations.

Emerging Trends in DevOps Testing

- **Shift-Right Testing:** Emphasizing testing in production environments to gather real-world feedback and insights, complementing traditional shift-left testing approaches.
- **Chaos Engineering:** Introducing controlled experiments to uncover weaknesses and failure modes in distributed systems, enhancing resilience and reliability.
- **AI-driven Testing:** Leveraging AI and machine learning to automate test generation, execution, and analysis, improving testing efficiency and effectiveness.
- **Containerization and Microservices Testing:** Focusing on testing strategies and tools tailored for containerized and microservices-based architectures to ensure compatibility, scalability, and performance.
- **Compliance as Code:** Implementing compliance checks and governance controls as code to automate and enforce regulatory compliance throughout the DevOps pipeline.



Q&A